Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 1
Introduction to Algorithms
Εισαγωγή στους Αλγόριθμους

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

What is an Algorithm?

- An **algorithm** (**αλγόριθμος**) is a well-defined computational procedure that
  - takes a value, or a set of values, as **input** (**είσοδος**)
  - performs a sequence of computational steps
  - and produces some value, or set of values, as **output** (**έξοδος**)
- Algorithms are used for solving **computational problems** (**υπολογιστικά προβλήματα**)

Description of a Problem vs. Instance of a Problem

- A computational problem is usually described in terms of its input and desired output
- Example: **sorting problem** (**ταξινόμηση**)
  - Input: A sequence of n numbers $(a_1, a_2, \dots, a_n)$
  - Output: A permutation $(a_1', a_2', \dots, a_n')$ of the input sequence such that $a_1' \leq a_2' \leq \cdots \leq a_n'$
- An **instance** (**στιγμιότυπο**) of a problem is some particular input
- Example: the sequence $(31, 41, 59, 26, 41, 58)$ is an instance of the sorting problem

A Correct Algorithm

- An algorithm is correct if, for every instance, it **halts** (**τερματίζει**) with the **correct output**
- We say that a correct algorithm **solves** (**επιλύει**) a computational problem

Why study algorithms?

- To learn methods for solving well known problems
- BUT, most importantly...
- To learn how to
  - **design** your own algorithms
  - **prove** their correctness
  - **analyse** their performance
- Is it worth also studying problems that are likely to have no efficient solution? (e.g., **NP**-complete problems)
- Yes, it is because
  - you don't spend fruitless time trying to find efficient solutions
  - you might find an **approximate** (**προσεγγιστική**) solution
  - You might be able to solve easily a variant of the problem

Algorithm Efficiency

- *Question*: consider a problem P and two algorithms A and B that both solve P but using different methodology. Algorithm A solves an instance of P in 10 sec. while algorithm B solves the same instance on the same computer in 10 min. Which one is better? Why?
- The example above demonstrates that it is not a good idea to measure algorithm efficiency simply in terms of seconds or minutes, since the required time:
  - depends on the size of the input
  - depends on the computer where the algorithm is run
  - could vary even for inputs of the same size

Algorithm Efficiency

- The efficiency of an algorithm is expressed in terms of the number of **operations** (**εντολές**) it performs as a function of the input size
  - e.g. $4n^2 + 8n, 5n \log n$, etc.
- Why is that better than counting seconds?
- Consider the previous example and let the algorithm A perform $2n^2$ operations for input size n, while algorithm B performs $50n \log_{10} n$ operations respectively. Notice:
  - for input size $n = 10$, A requires 200 operations and B 500
  - for $n = 1000$, A requires 2,000,000 ops while B only 150,000
  - how about if $n = 1{,}000{,}000$?

Exercises

- Consider a computer that can perform 10^6 operations per second. Calculate how much time (in seconds) will be required for an algorithm with the following complexities

Complexity	n=10	n=20	n=50
n	10^{-5}	2×10^{-5}	5×10^{-5}
n^2	10^{-4}	4×10^{-4}	25×10^{-4}
n^5	0.1	3.2	312.5
2^n	$\sim 10^{-3}$	~ 1	~35.7 years

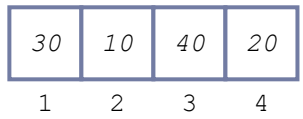
Algorithm Example

- Consider the problem of sorting
  - Input: A sequence of n numbers $(a_1, a_2, \ldots, a_n)$ (called the **keys**)
  - Output: A permutation $(a_1', a_2', \ldots, a_n')$ such that $a_1' \leq a_2' \leq \cdots \leq a_n$
  - One algorithm for solving the problem is **Insertion-Sort** (**Ενθετική Ταξινόμηση**)

```
InsertionSort(A, n) {
   for i = 2 to n {
     key = A[i]
     j = i - 1
     while (j > 0) and (A[j] > key) {
       A[j+1] = A[j]
       j = j - 1
     }
     A[j+1] = key
   }
}
```

Question: Why do we use pseudocode and not just a programming language, e.g., Java or C?

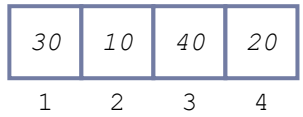
Example: Running Insertion Sort



$i = \varnothing \quad j = \varnothing \quad key = \varnothing$
$A[j] = \varnothing \quad A[j+1] = \varnothing$

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

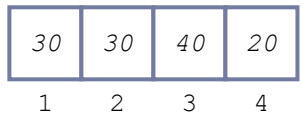
Example: Running Insertion Sort



i = 2 j = 1 key = 10
A[j] = 30 A[j+1] = 10

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

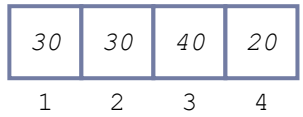

Example: Running Insertion Sort



i = 2 j = 1 key = 10
A[j] = 30 A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

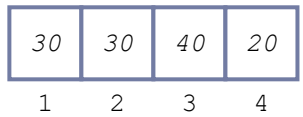
Example: Running Insertion Sort



i = 2 j = 1 key = 10
A[j] = 30 A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

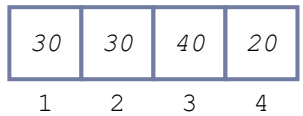

Example: Running Insertion Sort



i = 2 j = 0 key = 10
A[j] = ∅ A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

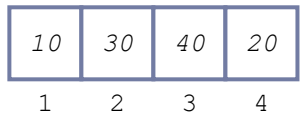
Example: Running Insertion Sort



i = 2 j = 0 key = 10
A[j] = ∅ A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

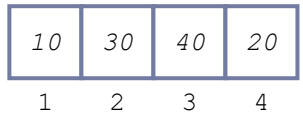

Example: Running Insertion Sort



i = 2 j = 0 key = 10
A[j] = ∅ A[j+1] = 10

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

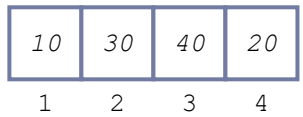
Example: Running Insertion Sort



$i = 3 \quad j = 0 \quad key = 10$
$A[j] = \varnothing \qquad A[j+1] = 10$

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

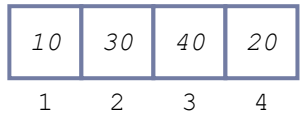

Example: Running Insertion Sort



$i = 3 \quad j = 0 \quad key = 40$
$A[j] = \varnothing \qquad A[j+1] = 10$

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

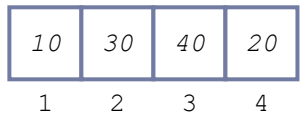
Example: Running Insertion Sort



i = 3 j = 0 key = 40
A[j] = ∅ A[j+1] = 10

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

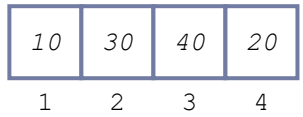

Example: Running Insertion Sort



i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

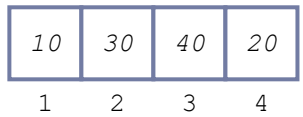
Example: Running Insertion Sort



i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

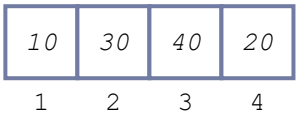

Example: Running Insertion Sort



i = 3 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

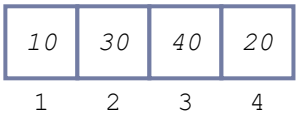
Example: Running Insertion Sort



i = 4 j = 2 key = 40
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

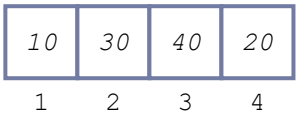

Example: Running Insertion Sort



i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

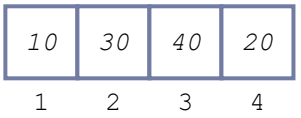
Example: Running Insertion Sort



i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

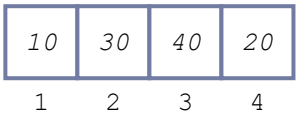

Example: Running Insertion Sort



i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 20

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

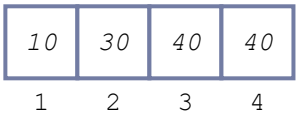
Example: Running Insertion Sort



i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 20

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

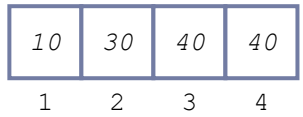

Example: Running Insertion Sort



i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

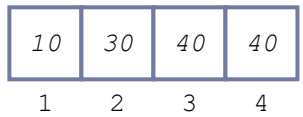
Example: Running Insertion Sort



i = 4 j = 3 key = 20
A[j] = 40 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

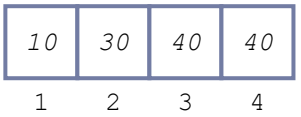

Example: Running Insertion Sort



i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

Example: Running Insertion Sort

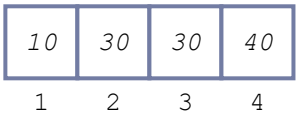


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 40



```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```


Example: Running Insertion Sort

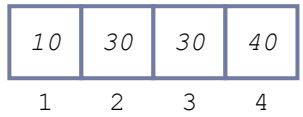


i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 30



```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

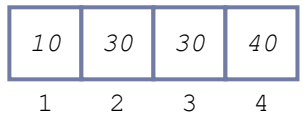
Example: Running Insertion Sort



i = 4 j = 2 key = 20
A[j] = 30 A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

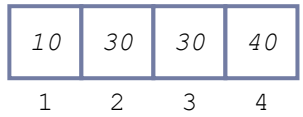

Example: Running Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

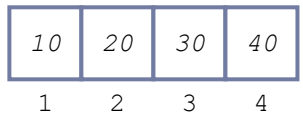
Example: Running Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 30

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

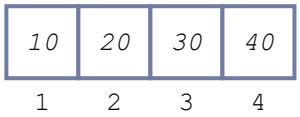

Example: Running Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 20

```
InsertionSort(A, n) {
  for i = 2 to n {
       key = A[i]
       j = i - 1;
       while (j > 0) and (A[j] > key) {
             A[j+1] = A[j]
             j = j - 1
       }
       A[j+1] = key
  }
}
```

Example: Running Insertion Sort



i = 4 j = 1 key = 20
A[j] = 10 A[j+1] = 20

```
InsertionSort(A, n) {
  for i = 2 to n {
      key = A[i]
      j = i - 1;
      while (j > 0) and (A[j] > key) {
            A[j+1] = A[j]
            j = j - 1
      }
      A[j+1] = key
  }
}
```

Done!

Exercise

- Apply Insertion-Sort on the following list of numbers and demonstrate how the list changes after each iteration of the algorithm

5 1 4 9 6 8 2

1 5 4 9 6 8 2
1 4 5 9 6 8 2
1 4 5 9 6 8 2
1 4 5 6 9 8 2
1 4 5 6 8 9 2
1 2 4 5 6 8 9

Showing Correctness through Loop Invariants

- We show correctness of an algorithm by stating and proving three things regarding its **loop invariant** (**αναλλοίωτη συνθήκη βρόχου**)
  - **Initialization** (**Αρχικός Έλεγχος**): It is true prior to the first iteration of the loop
  - **Maintenance** (**Έλεγχος Διατήρησης**): If it is true before an iteration, it remains true after it
  - **Termination** (**Επιβεβαίωση Αποτελέσματος**): At termination, the invariant gives us a useful property that proves the correctness of the algorithm
- Notice the similarity to *mathematical induction* (*μαθηματική επαγωγή*)!

Correctness of Insertion-Sort

- **Loop invariant**

  "At the beginning of each iteration of the **`for`** loop, the subarray $A[1..i-1]$ consists of the elements that were originally in $A[1..i-1]$ but in sorted order"
- **Initialization**
  - Is the invariant true before the first iteration ($i = 2$)?
- **Maintenance**
  - If the invariant is true at the kth step, will it also be at the $k+1$ step?
- **Termination**
  - What happens when the loop terminates? (Notice that $i = n+1$)

Analysing Insertion-Sort

- The **running time** (**χρόνος εκτέλεσης**) of an algorithm is the number of primitive operations or "steps" executed
- Let c_k denote the execution time required at line k
- Let t_i denote the number of times the while loop test is executed for iteration i

```
   InsertionSort(A, n) {
1:   for i = 2 to n {
2:      key = A[i]
3:      j = i - 1;
4:      while (j > 0) and (A[j] > key) {
5:         A[j+1] = A[j]
6:         j = j - 1
        }
7:      A[j+1] = key
    }
   }
```

The running time is:

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^{n} t_i + c_5 \sum_{i=2}^{n} (t_i - 1) + c_6 \sum_{i=2}^{n} (t_i - 1) + c_7(n-1)$$

Analysing Insertion-Sort (cont'd)

- Notice that the running time of Insertion-Sort depends not only on the size, but also on how well sorted the list is at the beginning of the algorithm
  - Lines 5 and 6 will execute only if the conditions of Line 4 are true
- In the best case (which is...?)
  - Lines 5 and 6 will not execute at all
  - Thus, $T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)\, n - (c_2 + c_3 + c_4 + c_7)$
  - That is, $T(n) = an + b$, for some constants a and b, i.e. it is a **linear** (**γραμμική**) function of n

Analysing Insertion-Sort (cont'd)

- In the worst case (which is...?)
  - Lines 4, 5 and 6 will execute the maximum number of times
  - Recall that:

$$\sum_{i=2}^{n} i = \left(\sum_{i=1}^{n} i\right) - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{i=2}^{n} (i-1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

  - By substituting in the formula of $T(n)$, we get $T(n) = an^2 + bn + c$ for constants a, b and c; i.e., it is a **quadratic** (**τετραγωνική**) function of n

Kinds of Analysis

- **Worst-case** (**Χειρότερης περίπτωσης**)
  - The most common type of analysis
  - It gives an **upper bound** (**άνω φράγμα**) on the running time for any input
- **Average-case** (**Αναμενόμενης περίπτωσης**)
  - We estimate the expected time of the algorithm over all inputs of size n
  - This is useful when the average-case running time is significantly lower than the worst case
  - We need to make an assumption about the statistical distribution of the inputs
- **Best-case** (**Καλύτερης περίπτωσης**)
  - Not really used

Exercise

- The following algorithm checks to see if the input array contains any value twice

```
duplicate = FALSE
for i=1 to n {
   for j=1 to n {
      if  (i != j) and (A[i] = A[j]) {
         duplicate = TRUE;
      }
   }
}
return duplicate;
```

  - Compute the worst-case running time of the algorithm.
  - Do you see any way of improving the algorithm?
  - What would be the running time in this case?